

Nakijk 01_03

NAKIJK · Hoofdstuk 01 · Twee routeparameters in één route · bij Oefening_01_03.md · tweede ronde, 4 oktober 2026

Opmerking

Dit is een nakijk van jouw code, geen oplossing. Er staat geen verbeterde code in. Per fout lees je waar ze zit, wat je API nu antwoordt, waarom, en in welke richting je moet zoeken.

Dit is de **tweede ronde**. Ze vervangt de eerste. Bovenaan zie je wat er met de zes fouten van de eerste ronde gebeurd is.

In één oogopslag

Je code is opnieuw getest met alle tien de voorbeelden uit de opgave, met `/api` ervoor. **Negen** geven nu letter voor letter het gevraagde antwoord. Het tiende, het kantoorvoorbeeld uit punt 5, nog niet.

#	Fout uit de eerste ronde	Status
1	namen in de socketroute ≠ namen van de parameters	opgelost
2	namen in de voedingsroute ≠ namen van de parameters	opgelost
3	slotenroute begon met <code>/</code> , en de namen verschilden	opgelost
4	montage had de verkeerde socket	opgelost
5	sloten en modules in de omgekeerde volgorde doorgegeven	nog open
6	de naam van de build stond vooraan in het antwoord	opgelost

A. Wat nog een verkeerd antwoord geeft

Fout 5 — sloten en modules nog altijd omgewisseld

Waar. De laatste regel van `InformatieBuild`, waar je `OpvragenGeheugen` oproept.

Verzoek	Je API antwoordt nu	De opgave vraagt
<code>GET /api/compatibiliteit/build/kantoor/LGA1700/450/4</code>	... Er blijven nog 2 sloten vrij.	... Er passen maar 2 modules in dit moederbord.
<code>GET /api/compatibiliteit/build/gaming/AM5/600/4</code>	... Alle sloten worden gebruikt.	... Alle sloten worden gebruikt. (toevallig juist)

Waarom. Je hebt de variabele in je `switch` hernoemd naar `sloten`, en dat is precies wat er moest gebeuren. Maar de volgorde in de oproep is gebleven. `OpvragenGeheugen` verwacht eerst `aantalSloten` en dan `aantalModules`. Jij geeft eerst `aantalModules` en dan `sloten`. De namen staan nu wel juist, maar op elkaars plaats: je API rekent dus met 4 sloten en 2 modules, terwijl de klant 4 modules wil in een moederbord met 2 sloten.

Bij `gaming` zie je het niet, omdat 4 en 4 toevallig gelijk zijn. Hoe groter het verschil tussen het aantal modules en het aantal sloten, hoe vreemder het antwoord wordt.

Richting. Lees de oproep en de kop van `OpvragenGeheugen` naast elkaar, argument per argument. Een methode-oproep koppelt op **volgorde**, niet op naam: het eerste argument gaat altijd naar de eerste parameter.

Twee manieren om dit voortaan te vermijden

- Roep in punt 5 je **hulpmethodes** op in plaats van de endpoints. Nu ze `private static` zijn, ligt dat nog meer voor de hand. Hun parameters heten kort en duidelijk `sloten` en `modules`, zodat een omwisseling sneller opvalt.
- C# laat je bij een oproep ook de naam van de parameter voor het argument zetten, gevolgd door een dubbele punt. Dan maakt de volgorde niet meer uit, en leest de oproep als een zin.

B. Werkt, maar wijkt af van de opgave

Wat	Waarom het toch aandacht vraagt
Je route begint met <code>api/</code> .	De voorbeelden in de opgave staan zonder <code>api</code> . Verdedigbaar, de cursus raadt <code>api/</code> zelfs aan. Zet het dan wel voor elk voorbeeld als je test.
Bij één vrij slot zegt je API <i>slot</i> in plaats van <i>sloten</i> .	Beter Nederlands, maar de opgave vraagt letterlijk <i>Er blijven nog {aantal} sloten vrij</i> . Bij een examen houd je je aan de opgave.

C. Opkuis, zonder invloed op het antwoord

- In de voedingsroute heten de parameters nog `{wat}` en `{gebruik}`. Die namen ziet wie je API in Scalar opent. `wattage` en `verbruik`, zoals in de opgave en in je eigen hulpmethode, zeggen meer.
- `OpvraagVerbruik` en `VerbruikVoeding` geven geen verbruik terug, maar een voedingsadvies. Een naam die zegt wat de methode teruggeeft, leest makkelijker.
- Een `$` voor een tekst zonder `{...}` erin, zoals *De processor past op dit moederbord.*, doet nog altijd niets.
- Tussen elk attribuut en zijn methode staat een lege regel. Dat werkt, maar meestal staat het attribuut er rechtstreeks boven, zodat je ziet dat ze bij elkaar horen.

D. Wat beter is dan in de eerste ronde

- De namen in elke route en de parameters van elk endpoint komen overeen. Daardoor geven punt 2, 3 en 4 alle acht de juiste antwoorden.
- De schuine streep vooraan de slotenroute is weg: het endpoint staat onder `/api/compatibiliteit`.
- De variabelen in je `switch` heten nu `socketMoederbord`, `verbruik` en `sloten`. Ze zeggen wat erin zit, en dat maakt fout 5 nu zichtbaar in één regel.
- Montage heeft de juiste socket, en de naam van de build staat niet meer in het antwoord.
- Je hulpmethodes zijn `private static`. Een goede keuze: ze gebruiken niets van het controllerobject zelf (leerboek, hoofdstuk 24).
- `GebruikteSloten` begint met een hoofdletter, en de overbodige usings zijn weg.

E. Zelf nakijken

Gebruik enkel de voorbeelden uit `Oefening_01_03.md`, met `/api` ervoor.

Na het oplossen van	Doe deze verzoeken	Je moet krijgen
fout 5	het kantoorvoorbeeld uit punt 5	het antwoord letter voor letter gelijk aan de opgave
daarna	alle tien de voorbeelden: twee uit punt 2, drie uit punt 3, drie uit punt 4 en twee uit punt 5	tien keer exact het gevraagde antwoord

Klopt alles, leg je endpoints dan ook nog naast de nakijkpunten in `Oplossing_01_03.md`.